

TP 6 — GitOps, Observabilité SRE & Chaos Engineering

Cours 6 | Développer pour le Cloud | YNOV Campus Montpellier — Master 2

Date : 23/06/2026 | Durée TP : 3h30 | Plateforme : Google Cloud Platform

Contexte entreprise — FraudGuard (suite du TP5)

Le pipeline de détection de fraude FraudGuard est en production depuis trois mois. Il traite désormais **5 millions de transactions par jour** et détecte en moyenne **2 400 alertes/jour**. Cependant, deux incidents majeurs ont secoué l'équipe :

- **Incident #INC-042 (mars 2026)** : un ingénieur a appliqué un `kubectl apply` direct en production pour corriger un bug "rapide". Le manifeste contenait une typo (`replicas: 0`). Le `fraud-detector` est resté arrêté **47 minutes** avant détection — 180 transactions frauduleuses non détectées, perte estimée 38 000€.
- **Incident #INC-051 (avril 2026)** : un déploiement Helm d'une nouvelle version du `fraud-detector` a introduit une régression. Les alertes CRITICAL n'étaient plus envoyées. Détection après **2h13** par un client se plaignant d'un débit frauduleux non bloqué. SLA contractuel violé : pénalité de **125 000€**.

La direction a tranché : FraudGuard doit migrer vers une approche **GitOps stricte** (zéro `kubectl` manuel en prod), déployer une **stack d'observabilité SRE complète** (Prometheus, Grafana, Loki, Tempo) avec des SLO/SLI mesurables, et valider la résilience par du **Chaos Engineering** avant chaque release majeure. Vous êtes le/la **Platform & SRE Engineer** chargé(e) de cette transformation.

Prérequis validés (Cours 5) :

- Cluster GKE `fraudguard-cluster` opérationnel
- Pipeline Kafka Streams (producer + fraud-detector + alert-handler) déployé
- Stack monitoring basique (kube-prometheus-stack) installée
- DAGs Airflow de reporting fonctionnels

Objectifs de ce TP :

- Déployer **ArgoCD** sur GKE et configurer le pattern **App of Apps** pour FraudGuard
- Implémenter un déploiement progressif **Canary** avec **Argo Rollouts** et analyse automatique
- Compléter la stack d'observabilité avec **Loki** (logs) et **Tempo** (traces) — LGTM Stack
- Définir des **SLO/SLI** mesurables et calculer un **Error Budget** opérationnel
- Exécuter des **expériences Chaos** avec Chaos Mesh (PodChaos, NetworkChaos)
- Préparer une **architecture multi-cloud** (GCP + AWS) pour le Disaster Recovery

Livrables attendus :

- ☐ Dépôt Git `fraudguard-gitops` avec la structure App of Apps complète
- ☐ ArgoCD synchronisant automatiquement l'infra + l'applicatif depuis Git
- ☐ Rollout Canary du `fraud-detector` v2 avec AnalysisTemplate Prometheus
- ☐ Dashboard Grafana SLO avec Error Budget en temps réel
- ☐ Trace distribuée Producer → Detector → Alert visible dans Tempo
- ☐ Rapport Chaos Engineering (Markdown) avec 3 expériences exécutées et analysées

- ☐ README.md décrivant l'architecture GitOps + Multi-cloud cible

Partie 1 — GitOps avec ArgoCD : Mettre fin aux `kubectl` manuels

Rappel du cours : GitOps repose sur 4 principes — **Déclaratif** (état désiré dans Git), **Source de vérité** (Git = unique source), **Pull-based** (l'agent tire depuis Git), **Immutabilité** (rollback par `git revert`). C'est l'inverse du modèle traditionnel CI/CD push où le pipeline pousse vers le cluster avec des *credentials exposés*.

1.1 — Installer ArgoCD sur GKE

```
# 1. Création du namespace dédié
kubectl create namespace argocd

# 2. Installation du manifeste stable ArgoCD
kubectl apply -n argocd -f \
  https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

# 3. Attendre que les pods soient prêts (peut prendre 3-5 minutes)
kubectl wait --for=condition=Ready pods --all -n argocd --timeout=300s

# 4. Exposer l'UI ArgoCD via LoadBalancer
kubectl patch svc argocd-server -n argocd \
  -p '{"spec": {"type": "LoadBalancer"}}' # LoadBalancer

# 5. Récupérer l'IP publique
ARGOCD_IP=$(kubectl get svc argocd-server -n argocd \
  -o jsonpath='{.status.loadBalancer.ingress[0].ip}')

# 6. Récupérer le mot de passe initial admin
ARGOCD_PWD=$(kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath="{.data.password}" | base64 -d)

echo "ArgoCD UI : https://${ARGOCD_IP}"
echo "Login    : admin / ${ARGOCD_PWD}"
```

Installez ensuite le CLI `argocd` localement :

```
# macOS
brew install argocd

# Connexion CLI
argocd login ${ARGOCD_IP} --username admin --password ${ARGOCD_PWD} --insecure
```

1.2 — Structurer le dépôt GitOps `fraudguard-gitops`

Bonne pratique : séparer le code applicatif (dépôt `fraudguard-app`) des manifestes Kubernetes (dépôt `fraudguard-gitops`). Cela évite que chaque commit de code déclenche un sync ArgoCD et permet de versionner indépendamment l'infra.

Créez le dépôt Git suivant (GitHub ou GitLab) :

```
fraudguard-gitops/
├── bootstrap/
│   └── root-app.yaml          # Root Application (App of Apps)
├── apps/
│   ├── infrastructure/      # Wave 0 – Plateforme
│   │   ├── kafka/
│   │   │   └── kafka-app.yaml
│   │   ├── monitoring/
│   │   │   └── monitoring-app.yaml
│   │   └── argo-rollouts/
│   │       └── rollouts-app.yaml
│   └── business/            # Wave 2 – Applicatif métier
│       ├── tx-producer-app.yaml
│       ├── fraud-detector-app.yaml
│       └── alert-handler-app.yaml
└── manifests/
    ├── infrastructure/
    │   ├── kafka/
    │   │   └── fraudguard-cluster.yaml
    │   ├── monitoring/
    │   │   ├── kustomization.yaml
    │   │   └── values-prometheus.yaml
    │   └── argo-rollouts/
    │       └── install.yaml
    ├── business/
    │   ├── tx-producer/
    │   │   ├── deployment.yaml
    │   │   └── kustomization.yaml
    │   ├── fraud-detector/
    │   │   ├── rollout.yaml
    │   │   ├── service.yaml
    │   │   ├── analysis-template.yaml
    │   │   └── kustomization.yaml
    │   └── alert-handler/
    │       └── deployment.yaml
```

Initialisez le dépôt localement :

```
mkdir -p fraudguard-gitops && cd fraudguard-gitops
git init
git remote add origin https://github.com/<VOTRE-ORG>/fraudguard-gitops.git
```

1.3 — Créer la Root Application (pattern App of Apps)

Le pattern **App of Apps** consiste à créer une seule **Application** ArgoCD racine qui déploie d'autres **Application**. Cela permet de bootstrapper tout l'environnement avec un seul `kubectl apply` initial. Ensuite, tout passe par Git.

Créez `bootstrap/root-app.yaml` :

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: fraudguard-root
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default
  source:
    repoURL: https://github.com/<VOTRE-ORG>/fraudguard-gitops.git
    targetRevision: main
    # Chemin contenant TOUTES les Applications enfants
    path: _____ # apps
    directory:
      recurse: true # Découvrir les Applications dans tous les sous-dossiers
  destination:
    server: https://kubernetes.default.svc
    namespace: argocd
  syncPolicy:
    automated:
      prune: _____ # true (supprimer les ressources retirées du Git)
      selfHeal: _____ # true (corriger automatiquement les drifts)
    syncOptions:
      - CreateNamespace=true

```

Créez une Application enfant pour le `fraud-detector` dans `apps/business/fraud-detector-app.yaml` :

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: fraud-detector
  namespace: argocd
  annotations:
    # Sync Wave : ordre de déploiement (négatif = avant, positif = après)
    # Infrastructure = wave 0, Plateforme = wave 1, Business = wave 2
    argocd.argoproj.io/sync-wave: "_____" # "2"
spec:
  project: default
  source:
    repoURL: https://github.com/<VOTRE-ORG>/fraudguard-gitops.git
    targetRevision: main
    path: manifests/business/fraud-detector
  destination:
    server: https://kubernetes.default.svc
    namespace: fraudguard
  syncPolicy:
    automated:
      prune: true
      selfHeal: true

```

```
syncOptions:
  - CreateNamespace=true
```

Pushez le dépôt et appliquez UNIQUEMENT la Root Application :

```
git add . && git commit -m "feat: initial GitOps structure"
git push -u origin main

# Le SEUL kubectl apply manuel autorisé : la Root App
kubectl apply -f bootstrap/root-app.yaml

# Vérifier que ArgoCD déploie tout
argocd app list
argocd app get fraudguard-root
```

Question : Pourquoi le sync wave 0 est-il attribué à kafka et 2 au fraud-detector ? Que se passerait-il si on déployait tout simultanément (sans sync wave) ?

Réponse :

1.4 — Tester le Self-Heal d'ArgoCD

Le **Self-Heal** est la fonctionnalité-clé qui empêche les incidents type INC-042. Toute modification manuelle sur le cluster est automatiquement annulée en moins de 3 minutes.

```
# 1. Vérifier l'état initial du fraud-detector
kubectl get deployment fraud-detector -n fraudguard -o jsonpath='{.spec.replicas}'
# → doit afficher 1

# 2. SIMULER l'incident INC-042 : modifier manuellement les replicas à 0
kubectl scale deployment fraud-detector -n fraudguard --replicas=0

# 3. Vérifier immédiatement (avant que ArgoCD ne réagisse)
kubectl get deployment fraud-detector -n fraudguard -o jsonpath='{.spec.replicas}'
# → 0 (drift créé)

# 4. Attendre 3 minutes maximum (intervalle de réconciliation ArgoCD par défaut)
sleep 180

# 5. Vérifier que ArgoCD a corrigé automatiquement
kubectl get deployment fraud-detector -n fraudguard -o jsonpath='{.spec.replicas}'
# → 1 (self-heal a restauré l'état désiré du Git)

# 6. Voir l'événement dans ArgoCD
argocd app history fraud-detector
```

Complétez le tableau d'observation :

Action	État avant	État après 3 min	Self-Heal détecté ?
--------	------------	------------------	---------------------

kubectl scale --replicas=0	_____	_____	_____
kubectl delete pod fraud-detector-xxx	_____	_____	_____ (cas particulier)
kubectl edit configmap kafka-config (changer une valeur)	_____	_____	_____

1.5 — Déploiement Canary avec Argo Rollouts

Argo Rollouts étend Kubernetes avec une ressource `Rollout` qui remplace `Deployment` et ajoute des stratégies *Canary* et *Blue-Green*. Pour *FraudGuard*, on déploiera v2 du `fraud-detector` progressivement (20% → 40% → 100%) avec une analyse *Prometheus* à chaque étape.

Installez Argo Rollouts (via GitOps, ajoutez `apps/infrastructure/argo-rollouts/`) :

```
# Plugin kubectl
brew install argoproj/tap/kubectl-argo-rollouts

# Une fois ajouté au Git, ArgoCD installera Argo Rollouts automatiquement
# Vérification :
kubectl get pods -n argo-rollouts
```

Créez `manifests/business/fraud-detector/rollout.yaml` :

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: fraud-detector
  namespace: fraudguard
spec:
  replicas: 4
  strategy:
    canary:
      # Service qui pointe TOUJOURS vers la version stable
      stableService: fraud-detector-stable
      # Service qui pointe vers la nouvelle version (Canary)
      canaryService: fraud-detector-canary
      steps:
        - setWeight: _____ # 20 (20% du trafic vers Canary)
        - pause: { duration: 2m } # Observer pendant 2 minutes
        - analysis:
            templates:
              - templateName: fraud-detector-success-rate
        - setWeight: _____ # 40
        - pause: { duration: 2m }
        - analysis:
            templates:
              - templateName: fraud-detector-success-rate
        - setWeight: 80
        - pause: { duration: 1m }
```

```

    - setWeight: 100    # Promotion complète
selector:
  matchLabels:
    app: fraud-detector
template:
  metadata:
    labels:
      app: fraud-detector
  spec:
    containers:
      - name: fraud-detector
        image: europe-west9-docker.pkg.dev/[PROJECT_ID]/tp2-registry/fraudguard-
streams:v2
        env:
          - name: KAFKA_BOOTSTRAP_SERVERS
            value: "fraudguard-kafka-kafka-
bootstrap.fraudguard.svc.cluster.local:9092"
        resources:
          requests:
            cpu: "500m"
            memory: "512Mi"
          limits:
            cpu: "2"
            memory: "2Gi"

```

Créez `manifests/business/fraud-detector/analysis-template.yaml` :

```

# AnalysisTemplate : interroge Prometheus pour valider le Canary
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: fraud-detector-success-rate
  namespace: fraudguard
spec:
  args:
    - name: service-name
      value: fraud-detector-canary
  metrics:
    - name: success-rate
      interval: 30s
      # Seuil : taux de succès doit rester > 99%
      successCondition: result[0] >= 0.99
      # Si la condition échoue 3 fois consécutivement → rollback automatique
      failureLimit: 3
      provider:
        prometheus:
          address: http://monitoring-kube-prometheus-prometheus.monitoring:9090
          # Métrique custom exposée par le fraud-detector (compteur Prometheus)
          # incrémenté à chaque transaction traitée, avec un label "status"
          # (success|error) et "version" (stable|canary).
          query: |

```

```

sum(rate(fraud_detection_processed_total{
  status="success",
  version="canary"
}[5m]))
/
sum(rate(fraud_detection_processed_total{
  version="canary"
}[5m]))

```

Déclencher le rollout en pushant un changement de tag d'image dans Git :

```

# Modifier l'image v1 → v2 dans rollout.yaml puis :
git add . && git commit -m "feat(fraud-detector): rollout v2 with canary"
git push

# Suivre le rollout en temps réel
kubectl argo rollouts get rollout fraud-detector -n fraudguard --watch

# Si l'AnalysisRun détecte une dégradation → rollback automatique
# Sinon → promotion automatique à 100% après les pauses

```

Question : Quelle est la différence fondamentale entre la stratégie **Canary** d'Argo Rollouts et la stratégie **Blue-Green** ? Dans quel cas FraudGuard devrait-elle préférer Blue-Green ?

Réponse :

Partie 2 — Observabilité SRE : la stack LGTM complète

Le **slogan SRE** : « You can't improve what you don't measure ». La stack LGTM (**L**oki, **G**rafana, **T**empo, **M**imir/Prometheus) couvre les 3 piliers de l'observabilité : **Logs**, **Métriques**, **Traces**. La corrélation entre les 3 (via `trace_id`) permet d'investiguer un incident en quelques minutes au lieu de plusieurs heures.

2.1 — Déployer Loki pour l'agrégation de logs

Note 2026 : le chart historique `grafana/loki-stack` est déprécié depuis 2023. On utilise désormais le chart officiel `grafana/loki` (mode `SingleBinary` pour le TP) et on déploie Promtail via le chart `grafana/promtail` séparé.

Créez `manifests/infrastructure/monitoring/loki-values.yaml` :

```

# Chart grafana/loki en mode SingleBinary (pour la prod : SimpleScalable ou
Distributed)
deploymentMode: SingleBinary
loki:
  auth_enabled: false
  commonConfig:
    replication_factor: 1
  schemaConfig:
    configs:
      - from: "2026-01-01"

```



```

    store: tsdb
    object_store: filesystem
    schema: v13
    index:
      prefix: index_
      period: 24h
  storage:
    type: 'filesystem'
singleBinary:
  replicas: 1
  persistence:
    size: 10Gi
# On désactive les composants non utilisés en SingleBinary
read:    { replicas: 0 }
write:   { replicas: 0 }
backend: { replicas: 0 }

```

Créez `manifests/infrastructure/monitoring/promtail-values.yaml` :

```

# Promtail: DaemonSet qui collecte les logs depuis chaque node Kubernetes
config:
  clients:
    - url: http://loki.monitoring.svc.cluster.local:3100/loki/api/v1/push

```

Ajoutez l'Application ArgoCD `apps/infrastructure/monitoring/loki-app.yaml` .

Important — multi-source ArgoCD (v2.6+) : pour référencer des `valueFiles` situés dans un autre dépôt Git (`$values/...`), il faut utiliser le bloc **`spec.sources:`** (pluriel) avec un `ref:` nommé, et **non** `spec.source:` (singulier) qui ne sait pas résoudre `$values` .

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: loki
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "0"
spec:
  project: default
  sources:
    # Source 1 : le chart Helm officiel Loki
    - repoURL: https://grafana.github.io/helm-charts
      chart: loki
      targetRevision: 6.16.0
      helm:
        valueFiles:
          - $values/manifests/infrastructure/monitoring/loki-values.yaml
    # Source 2 : le dépôt GitOps qui contient nos values (nommé "values")
    - repoURL: https://github.com/<VOTRE-ORG>/fraudguard-gitops.git
      targetRevision: main
      ref: values

```

```

destination:
  server: https://kubernetes.default.svc
  namespace: monitoring
syncPolicy:
  automated: { prune: true, selfHeal: true }
  syncOptions:
    - CreateNamespace=true
---
# Promtail (agent de collecte) déployé séparément
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: promtail
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "1" # Après Loki
spec:
  project: default
  sources:
    - repoURL: https://grafana.github.io/helm-charts
      chart: promtail
      targetRevision: 6.16.6
      helm:
        valueFiles:
          - $values/manifests/infrastructure/monitoring/promtail-values.yaml
    - repoURL: https://github.com/<VOTRE-ORG>/fraudguard-gitops.git
      targetRevision: main
      ref: values
  destination:
    server: https://kubernetes.default.svc
    namespace: monitoring
  syncPolicy:
    automated: { prune: true, selfHeal: true }

```

Une fois ArgoCD synchronisé, connectez Loki à Grafana :

Grafana UI → Configuration → Data Sources → Add → Loki
 URL : http://loki.monitoring.svc.cluster.local:3100

2.2 — Requêtes LogQL utiles pour FraudGuard

Dans Grafana → Explore → Loki, testez ces requêtes :

```

# 1. Tous les logs du fraud-detector
{namespace="fraudguard", app="_____"} # fraud-detector

# 2. Filtrer les alertes CRITICAL uniquement
{namespace="fraudguard", app="fraud-detector"} |= "CRITICAL"

# 3. Compter les alertes par type sur les 5 dernières minutes (Metric Query)
sum by (alert_type) (

```

```

count_over_time(
  {namespace="fraudguard", app="alert-handler"}
  | json
  | alert_type != ""
  [5m]
)
)

# 4. Détecter les erreurs Kafka (regex)
{namespace="fraudguard"} |~ "(?i)(error|exception|failed)"

# 5. P99 de la latence de traitement (extraite des logs JSON)
quantile_over_time(0.99,
  {namespace="fraudguard", app="fraud-detector"}
  | json
  | unwrap processing_latency_ms [5m]
)

```

Question : Le cours mentionne d'éviter les labels à haute cardinalité (IP utilisateur, Request ID) dans l'index Loki. Pourquoi est-ce critique pour la performance ? Quelle est l'alternative pour pouvoir tout de même rechercher par Request ID ?

Réponse :

2.3 — Déployer Tempo pour le tracing distribué

Le but : visualiser **une seule transaction de fraude** depuis le `tx-producer` jusqu'au `alert-handler`, avec la latence de chaque étape.

Ajoutez l'Application ArgoCD `apps/infrastructure/monitoring/tempo-app.yaml` :

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: tempo
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "0"
spec:
  project: default
  source:
    repoURL: https://grafana.github.io/helm-charts
    chart: tempo
    targetRevision: 1.10.1
  helm:
    values: |
      tempo:
        storage:
          trace:
            backend: local
            local:

```

```

        path: /var/tempo
receivers:
  otlp:
    protocols:
      grpc: { endpoint: 0.0.0.0:4317 }
      http: { endpoint: 0.0.0.0:4318 }
destination:
  server: https://kubernetes.default.svc
  namespace: monitoring
syncPolicy:
  automated: { prune: true, selfHeal: true }

```

Instrumentez le `fraud-detector` avec OpenTelemetry. Créez `fraud-detection/streams/tracing.js` :

```

/**
 * Instrumentation OpenTelemetry pour le fraud-detector
 * Capture les spans : Kafka consume → analyze → publish alert
 *
 * API OpenTelemetry JS SDK ≥ 2.x (2026) :
 * - `Resource` est remplacé par `resourceFromAttributes()`
 * - `SemanticResourceAttributes` est remplacé par les constantes `ATTR_*`
 */
const { NodeSDK } = require('@opentelemetry/sdk-node');
const { OTLPTraceExporter } = require('@opentelemetry/exporter-trace-otlp-grpc');
const { resourceFromAttributes } = require('@opentelemetry/resources');
const {
  ATTR_SERVICE_NAME,
  ATTR_SERVICE_VERSION,
} = require('@opentelemetry/semantic-conventions');
const { getNodeAutoInstrumentations } = require('@opentelemetry/auto-instrumentations-node');

const sdk = new NodeSDK({
  resource: resourceFromAttributes({
    [ATTR_SERVICE_NAME]: 'fraud-detector',
    [ATTR_SERVICE_VERSION]: process.env.APP_VERSION || 'v2',
    'deployment.environment': process.env.ENV || 'production',
  }),
  traceExporter: new OTLPTraceExporter({
    // Envoyer les traces au Tempo via OTLP gRPC
    url: 'http://tempo.monitoring.svc.cluster.local:_____', // 4317
  }),
  instrumentations: [getNodeAutoInstrumentations({
    // Désactiver l'auto-instrumentation FS (trop bruyant)
    '@opentelemetry/instrumentation-fs': { enabled: false },
  })],
});

sdk.start();
console.log('[Tracing] OpenTelemetry initialisé – export vers Tempo');

```

```
module.exports = sdk;
```

Modifiez `fraud-detector.js` pour ajouter des spans custom :

```
require('./tracing'); // DOIT être le premier require
const { Kafka } = require('kafkajs');
const { trace } = require('@opentelemetry/api');

const tracer = trace.getTracer('fraud-detector', 'v2');

async function analyzeTransaction(transaction) {
  // Créer un span custom pour l'analyse de fraude
  return await tracer.startActiveSpan('analyze_transaction', async (span) => {
    span.setAttributes({
      'fraud.account_id': transaction.account_id,
      'fraud.tx_amount': transaction.amount,
      'fraud.tx_type': transaction.tx_type,
    });

    try {
      // ... logique de détection (inchangée) ...
      const alerts = []; // résultat de l'analyse

      span.setAttribute('fraud.alerts_count', alerts.length);
      if (alerts.length > 0) {
        span.setAttribute('fraud.alert_severity', alerts[0].severity);
        // Marquer le span comme "interessant" pour le sampling Tempo
        span.setAttribute('sampling.priority', 1); // 1 (force la capture)
      }
      span.setStatus({ code: 1 }); // OK
      return alerts;
    } catch (err) {
      span.recordException(err);
      span.setStatus({ code: 2, message: err.message }); // ERROR
      throw err;
    } finally {
      span.end();
    }
  });
}
```

Connectez Tempo à Grafana :

Grafana UI → Configuration → Data Sources → Add → Tempo
URL : `http://tempo.monitoring:3200`

Dans Grafana → Explore → Tempo → Search :

- Service Name : `fraud-detector`
- Cliquer sur une trace pour voir le waterfall complet

Question : Dans une trace distribuée, vous voyez que le span `kafka.publish.fraud-alerts` prend 850ms (alors que le span `analyze_transaction` ne prend que 12ms). Comment investigueriez-vous cette anomalie ? Quel outil de Grafana permet de corréler ce span avec les logs Loki du broker Kafka au même instant ?

Réponse :

2.4 — Définir les SLO/SLI de FraudGuard

Rappel cours : *SLI* = "ce que je mesure" (latence, taux de succès). *SLO* = "ma cible interne" (99.5% de réussite). *SLA* = "engagement contractuel externe" (99.0%, sinon pénalité). L'**Error Budget** = 100% - *SLO* est la marge d'erreur acceptable sur une période donnée.

Complétez le tableau des SLO FraudGuard :

Service	SLI (indicateur mesurable)	SLO (cible interne)	SLA (contrat client)	Error Budget mensuel
tx-producer	Taux de transactions publiées avec succès dans Kafka	99.95%	99.9%	21,6 min / mois
fraud-detector	Latence P99 de détection (Kafka consume → alert publish)	< _____ ms	< _____ ms	_____
alert-handler	Taux d'alertes CRITICAL traitées en < 5s	_____ %	_____ %	_____
Système global	Disponibilité E2E (depuis l'IngressGateway)	99.95%	99.9%	_____ min

Créez la **Recording Rule** Prometheus pour calculer l'Error Budget en temps réel. Ajoutez à `manifests/infrastructure/monitoring/recording-rules.yaml` :

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: fraudguard-slo-rules
  namespace: monitoring
spec:
  groups:
    - name: fraudguard.slo
      interval: 30s
      rules:
        # Taux de succès du fraud-detector sur 5 minutes
        - record: fraudguard:detector_success_rate:5m
          expr: |
            sum(rate(kafka_consumer_records_consumed_total{
              consumer_group="fraud-detection-group"
            }[5m]))
            /
            sum(rate(kafka_consumer_records_total{
              consumer_group="fraud-detection-group"
```

```

    }[5m]))

# Error Budget RESTANT sur le mois en cours (fraction entre 0 et 1)
# Formule SRE : 1 - (taux d'erreur observé / taux d'erreur autorisé)
#   taux d'erreur observé = 1 - avg(success_rate sur 30j)
#   taux d'erreur autorisé = 1 - SLO (ex: 1 - 0.995 = 0.005)
# Résultat : 1.0 = budget plein ; 0.0 = budget épuisé ; négatif = SLO violé
- record: fraudguard:detector_error_budget_remaining:30d
  expr: |
    1 - (
      (1 - avg_over_time(
        fraudguard:detector_success_rate:5m[30d]
      ))
      /
      (1 - _____) # SLO du fraud-
detector (ex: 0.995)
    )

# Alerte : Error Budget < 20% restant
- alert: ErrorBudgetCritical
  expr: fraudguard:detector_error_budget_remaining:30d < 0.20
  for: 5m
  labels:
    severity: critical
    team: sre
  annotations:
    summary: "Error Budget fraud-detector < 20% – Feature Freeze recommandé"
    runbook: "https://runbooks.fraudguard.fr/error-budget"

```

Question : L'Error Budget restant est tombé à **5%** en milieu de mois. Selon les principes SRE, quelle décision opérationnelle doit prendre l'équipe ? Quel impact pour les Product Managers ?

Réponse :

2.5 — Dashboard Grafana SLO

Dans Grafana, créez un dashboard `FraudGuard SLO Overview` avec :

Panel 1 — Error Budget restant (Gauge)

```

Query   : fraudguard:detector_error_budget_remaining:30d * 100
Unit    : percent (0-100)
Seuils  : Red < 20, Orange < 50, Green >= 50

```

Panel 2 — Burn Rate (vitesse de consommation du budget)

```

Query   : (1 - fraudguard:detector_success_rate:5m) / (1 - 0.995)
Unit    : Burn rate multiplier (1 = consommation nominale, 14.4 = budget brûlé en 1
jour)
Seuils  : Alert > 14.4 sur 1h (page-able), > 6 sur 6h

```

Panel 3 — Latence P99 vs SLO

```
Query : histogram_quantile(0.99,
    sum by (le) (rate(fraud_detection_duration_seconds_bucket[5m]))
) * 1000
Seuil : Ligne horizontale à 500ms (SLO P99)
```

Panel 4 — Disponibilité E2E (Time series)

```
Query : avg_over_time(fraudguard:detector_success_rate:5m[1h]) * 100
```

Partie 3 — Chaos Engineering avec Chaos Mesh

Principe : on ne valide pas la résilience par la théorie, on la prouve en cassant délibérément des composants en environnement contrôlé. **Hypothèse stable** → **expérience** → **observation** → **apprentissage**. ⚠ Toujours en staging d'abord, avec un **blast radius limité**.

3.1 — Installer Chaos Mesh via GitOps

Ajoutez `apps/infrastructure/chaos-mesh/chaos-app.yaml` :

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: chaos-mesh
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "0"
spec:
  project: default
  source:
    repoURL: https://charts.chaos-mesh.org
    chart: chaos-mesh
    targetRevision: 2.6.3
  helm:
    values: |
      dashboard:
        create: true
        serviceType: LoadBalancer
      chaosDaemon:
        runtime: containerd
        socketPath: /run/containerd/containerd.sock
  destination:
    server: https://kubernetes.default.svc
    namespace: chaos-mesh
  syncPolicy:
    automated: { prune: true, selfHeal: true }
    syncOptions:
      - CreateNamespace=true
```

Accédez au dashboard Chaos Mesh :


```
CHAOS_IP=$(kubectl get svc chaos-dashboard -n chaos-mesh \
-o jsonpath='{.status.loadBalancer.ingress[0].ip}')
echo "Chaos Mesh UI : http://${CHAOS_IP}:2333"
```

3.2 — Expérience #1 : PodChaos — tuer un fraud-detector aléatoirement

Hypothèse stable : Si l'un des 4 réplicas du `fraud-detector` est tué, Kubernetes le redémarre en < 30s et le rebalancing des partitions Kafka assure qu'aucune transaction n'est perdue (à condition que `enable.auto.commit=false` et que les commits soient explicites).

Créez `chaos/experiments/pod-failure-detector.yaml` :

Note Chaos Mesh v2+ : depuis la version 2.0, le champ `scheduler.cron` a été retiré du `spec` des expériences ponctuelles. Pour exécuter une expérience de manière récurrente, on utilise une ressource `Schedule` qui wrappe le `PodChaos`.

```
apiVersion: chaos-mesh.org/v1alpha1
kind: Schedule
metadata:
  name: kill-fraud-detector
  namespace: fraudguard
spec:
  schedule: '@every 60s'           # Toutes les 60 secondes, un nouveau kill
  historyLimit: 5                 # Conserver l'historique des 5 dernières
  exécutions
  concurrencyPolicy: Forbid       # Pas de chevauchement (attendre la fin avant de
  relancer)
  type: PodChaos                 # Type de l'expérience encapsulée
  podChaos:
    # Type d'action : "pod-kill" supprime le pod, "pod-failure" le rend indisponible
    action: pod-kill              # pod-kill
    mode: one                     # Cibler UN seul pod (blast radius limité)
    selector:
      namespaces:
        - fraudguard
      labelSelectors:
        app: fraud-detector
    # Durée de l'effet d'UNE expérience (10s suffisent pour le kill)
    duration: '10s'
```

Avant de lancer l'expérience, mesurez le baseline :

```
# Métriques baseline (à noter dans le tableau)
# 1. Taux de succès du fraud-detector (depuis Grafana)
# 2. Latence P99 de détection
# 3. Consumer lag Kafka
```

Lancez l'expérience :

```
kubectl apply -f chaos/experiments/pod-failure-detector.yaml

# Observer en temps réel
kubectl get pods -n fraudguard -l app=fraud-detector -w
# (vous verrez des pods passer en Terminating puis Running cycliquement)
```

Complétez le tableau pendant et après l'expérience (5 minutes) :

Métrique	Baseline	Pendant Chaos	Après Chaos (récupération)	Verdict
Réplicas Running	4	_____	_____	_____
Taux de succès %	_____	_____	_____	_____
Latence P99 (ms)	_____	_____	_____	_____
Consumer lag	_____	_____	_____	_____
Alertes manquées	0	_____	_____	_____
MTTR (sec)	—	—	_____	< 30s attendu

3.3 — Expérience #2 : NetworkChaos — latence Kafka

Hypothèse stable : Si la latence réseau entre `fraud-detector` et Kafka passe de 1ms à 200ms, le système reste fonctionnel mais la latence P99 dépasse temporairement le SLO de 500ms. Le circuit breaker doit éviter une cascade de timeouts.

Créez `chaos/experiments/network-latency-kafka.yaml` :

```
apiVersion: chaos-mesh.org/v1alpha1
kind: NetworkChaos
metadata:
  name: latency-kafka-detector
  namespace: fraudguard
spec:
  action: _____ # delay
  mode: all # Sur TOUS les pods fraud-detector
  selector:
    namespaces:
      - fraudguard
    labelSelectors:
      app: fraud-detector
  delay:
    # Ajouter 200ms ± 50ms de latence sur tout le trafic sortant
    latency: '_____' # '200ms'
    correlation: '50'
    jitter: '50ms'
  # Cibler UNIQUEMENT le trafic vers Kafka (ne pas casser le trafic vers Prometheus)
  target:
    selector:
      namespaces:
```

```
- fraudguard
labelSelectors:
  strimzi.io/cluster: fraudguard-kafka
mode: all
duration: '3m'
```

Lancez et observez :

```
kubectl apply -f chaos/experiments/network-latency-kafka.yaml

# Dans Grafana, observer le panel "Latence P99 vs SL0"
# La ligne doit dépasser 500ms (SL0 violé) pendant 3 minutes
# Puis revenir sous 500ms après la fin de l'expérience
```

3.4 — Expérience #3 : StressChaos — saturation CPU

Hypothèse stable : Si le CPU d'un node fraud-detector est saturé à 95%, l'HPA (Horizontal Pod Autoscaler) doit déclencher la création de réplicas supplémentaires en moins de 2 minutes. Si l'HPA n'est pas configuré, le SLO de latence sera violé pendant toute la durée du stress.

Créez d'abord un HPA :

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: fraud-detector-hpa
  namespace: fraudguard
spec:
  scaleTargetRef:
    apiVersion: argoproj.io/v1alpha1
    kind: Rollout
    name: fraud-detector
  minReplicas: 4
  maxReplicas: 12
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: _____ # 70 (scale-up dès 70% CPU)
```

Puis l'expérience de stress :

```
apiVersion: chaos-mesh.org/v1alpha1
kind: StressChaos
metadata:
  name: cpu-stress-detector
  namespace: fraudguard
spec:
```

```
mode: one
selector:
  namespaces:
    - fraudguard
  labelSelectors:
    app: fraud-detector
stressors:
  cpu:
    workers: 2
    load: 95 # 95% de CPU consommé par le stress
duration: '5m'
```

```
kubectl apply -f hpa.yaml
kubectl apply -f chaos/experiments/cpu-stress.yaml

# Observer l'HPA en action
kubectl get hpa fraud-detector-hpa -n fraudguard -w
```

3.5 — Rapport Chaos Engineering

Rédigez `CHAOS_REPORT.md` dans le dépôt `fraudguard-gitops/` avec ce template :

```
# Rapport Chaos Engineering FraudGuard – Sprint 2026-Q2

## Synthèse exécutive
- Nombre d'expériences exécutées : 3
- Hypothèses validées : ___ / 3
- Régressions détectées : ___
- Recommandations critiques : ___

## Expérience 1 – PodChaos
- Hypothèse : MTTR < 30s
- Résultat observé : _____
- Hypothèse validée : OUI / NON
- Actions correctives : _____

## Expérience 2 – NetworkChaos
- Hypothèse : SLO temporairement violé mais pas de cascade
- Résultat observé : _____
- Actions correctives : _____

## Expérience 3 – StressChaos
- Hypothèse : HPA déclenche scale-up < 2 min
- Résultat observé : _____
- Actions correctives : _____

## Game Day suivant
- Date prévue : _____
- Scénario : panne complète de la zone europe-west9-a (NodeChaos)
```

Question : Pourquoi est-il dangereux d'exécuter une expérience Chaos directement en production sans mode: one ni duration limitée ? Donnez 2 sécurités supplémentaires à mettre en place avant un Game Day en production.

Réponse :

Partie 4 — Préparation Multi-cloud (DR cross-cloud)

Après les incidents et la pression réglementaire, FraudGuard doit pouvoir basculer vers **AWS** en cas de panne majeure de GCP. L'objectif n'est pas de faire tourner les deux clouds en actif/actif (trop cher), mais de garantir un **RTO < 15 minutes** vers AWS en cas de désastre GCP.

4.1 — Architecture cible Multi-cloud

Complétez le diagramme suivant en associant chaque service à son équivalent AWS :

Service GCP (Primaire)	Équivalent AWS (DR)	Mécanisme de synchronisation
GKE (cluster fraudguard)	_____	Manifestes identiques via GitOps
Cloud Storage (modèles ML)	_____	Sync via Storage Transfer Service
Firestore (alertes)	_____	CDC via Change Data Capture
Artifact Registry (images)	_____	Mirror via Skopeo cron job
Cloud DNS	_____	Failover record (TTL = 60s)
Cloud Load Balancing	_____	Global Accelerator avec health check

4.2 — Terraform module multi-provider

Créez terraform/modules/fraudguard-cluster/main.tf :

```
# Module abstrait multi-provider pour le cluster FraudGuard
variable "cloud_provider" {
  type = string
  validation {
    condition      = contains(["gcp","aws"], var.cloud_provider)
    error_message = "cloud_provider doit être 'gcp' ou 'aws'."
  }
}

variable "cluster_name" {
  type      = string
  default = "fraudguard-cluster"
}

variable "region" {
  type = string
  # europe-west9 pour GCP, eu-west-3 pour AWS
}
```

```

# Cluster GKE (si GCP)
resource "google_container_cluster" "fraudguard" {
  count    = var.cloud_provider == "gcp" ? 1 : 0
  name     = var.cluster_name
  location = var.region

  initial_node_count = 3
  node_config {
    machine_type = "e2-standard-4"
  }
}

# Cluster EKS (si AWS)
resource "aws_eks_cluster" "fraudguard" {
  count    = var.cloud_provider == "aws" ? 1 : 0 # aws
  name     = var.cluster_name
  role_arn = aws_iam_role.eks.arn

  vpc_config {
    subnet_ids = aws_subnet.eks[*].id
  }
}

output "cluster_endpoint" {
  value = var.cloud_provider == "gcp" ?
    google_container_cluster.fraudguard[0].endpoint :
    aws_eks_cluster.fraudguard[0].endpoint
}

```

Pour déployer le cluster DR sur AWS :

```

cd terraform/environments/dr-aws
terraform init
terraform apply -var="cloud_provider=aws" -var="region=eu-west-3"

```

Une fois le cluster EKS prêt, le même dépôt GitOps `fraudguard-gitops` peut le configurer (avec un ArgoCD installé sur EKS pointant vers le même Git).

4.3 — Failover DNS et procédure de bascule

Créez la procédure `RUNBOOK_DR.md` :

```

# Runbook Disaster Recovery – Bascule GCP → AWS

## Critères de déclenchement
- [ ] GCP région europe-west9 indisponible > 10 min (status.cloud.google.com)
- [ ] Health check Global LB échoue depuis > 5 min
- [ ] Décision validée par : SRE Lead + CTO

## Étapes (RTO cible : 15 min)

```

T+0min : Activation

1. Page SRE on-call (PagerDuty)
2. Notifier #incident-war-room sur Slack
3. Démarrer le bridge call

T+2min : Vérification cluster DR AWS

```
```bash
kubectl --context=eks-fraudguard-dr get nodes
kubectl --context=eks-fraudguard-dr get pods -n fraudguard
Le cluster EKS tourne en "warm standby" avec 0 réplica fraud-detector
```

## T+5min : Scale-up des workloads

```
ArgoCD déjà installé sur EKS → modifier le repo Git
git checkout -b dr-failover
sed -i 's/replicas: 0/replicas: 4/g' manifests/business/*/rollout.yaml
git commit -am "DR: scale-up workloads on AWS"
git push origin dr-failover

Merger immédiatement (skip review en mode incident)
gh pr create --base main --head dr-failover --title "[DR] Failover GCP→AWS"
gh pr merge --merge --auto
```

## T+10min : Bascule DNS

```
Modifier le record DNS pour pointer vers le LB AWS
aws route53 change-resource-record-sets \
 --hosted-zone-id Z123ABC \
 --change-batch file://dr-failover-dns.json
TTL configuré à 60s → propagation rapide
```

## T+15min : Vérification SLO

- Latence P99 < 500ms : \_\_\_\_\_
- Taux de succès > 99% : \_\_\_\_\_
- Alertes traitées : \_\_\_\_\_

**\*\*Question :\*\*** Le coût mensuel du cluster EKS DR en "warm standby" (avec 3 nodes minimum) est de **~450€**. La direction propose de passer en "cold standby" (cluster détruit, recréé en cas de DR avec Terraform). Quel impact sur le RTO ? Cette économie est-elle pertinente pour FraudGuard ?

Réponse :

---

```
Nettoyage Final – IMPORTANT
```

```
```bash
```

```
# 1. Supprimer les expériences Chaos
```

```
kubectl delete podchaos --all -n fraudguard
```

```
kubectl delete networkchaos --all -n fraudguard
```

```
kubectl delete stresschaos --all -n fraudguard
```

```
# 2. Supprimer les Applications ArgoCD (cascade supprime tout)
```

```
argocd app delete fraudguard-root --cascade
```

```
kubectl wait --for=delete application/fraudguard-root -n argocd --timeout=300s
```

```
# 3. Désinstaller ArgoCD
```

```
kubectl delete namespace argocd
```

```
# 4. Désinstaller la stack monitoring (Loki, Tempo, Prometheus)
```

```
kubectl delete namespace monitoring
```

```
# 5. Désinstaller Chaos Mesh
```

```
kubectl delete namespace chaos-mesh
```

```
# 6. Supprimer le cluster GKE primaire
```

```
gcloud container clusters delete fraudguard-cluster --region=europe-west9 --quiet
```

```
# 7. Supprimer le cluster EKS DR (si créé)
```

```
cd terraform/environments/dr-aws
```

```
terraform destroy -auto-approve
```

```
# 8. Vérification finale
```

```
kubectl config get-contexts
```

```
gcloud container clusters list
```

```
aws eks list-clusters --region eu-west-3
```

Récapitulatif — Compétences validées

- ☐ **GitOps** : ArgoCD installé, pattern App of Apps, Self-Heal, sync waves
- ☐ **Progressive Delivery** : Argo Rollouts Canary avec AnalysisTemplate Prometheus
- ☐ **Observabilité SRE** : stack LGTM complète (Loki, Grafana, Tempo, Prometheus)
- ☐ **SLO/SLI/Error Budget** : Recording Rules Prometheus + Dashboard Grafana
- ☐ **Chaos Engineering** : 3 expériences exécutées (Pod, Network, Stress) + rapport
- ☐ **Multi-cloud** : architecture DR GCP↔AWS, Terraform module abstrait, runbook

Livrables finaux à remettre

- ☐ URL du dépôt Git `fraudguard-gitops` (structure App of Apps complète)
- ☐ Capture d'écran : ArgoCD UI montrant toutes les applications "Healthy / Synced"
- ☐ Capture d'écran : rollout Canary avec les steps (20% → 40% → 100%) visibles
- ☐ Capture d'écran : Dashboard Grafana SLO avec Error Budget et Burn Rate
- ☐ Capture d'écran : Trace distribuée Tempo (producer → detector → alert)

- ☐ CHAOS_REPORT.md complété avec les 3 expériences
- ☐ RUNBOOK_DR.md complété avec la procédure de bascule
- ☐ README.md avec diagramme de l'architecture GitOps + Multi-cloud cible